

Frontend :

HTML + CSS + JS + Leaflet

Backend:

Node.js + Express

Database:

PostgreSQL + PostGIS

Structure :

gis-map-data/

|

| — index.html

| — app.js

| — styles.css

|

| — pages/

| | — site.html

|

| — js/

| | — mapLayers.js

| | — siteMarkers.js

| | — sectorTool.js

| | — siteDetails.js

|

| — data/

| | — india.geojson

| | — states.geojson

| | — districts.geojson

| | — taluks.geojson

| | — sites.json

| | — villages_sample.geojson

| | — instruments_sample.json

|

| — lib/

| | — leaflet/

| | | — leaflet.js

| | | — leaflet.css

| | | — images/

|

| — backend_later/

| | — server.js

| | — db.js

| | — routes/

| | — sql/

Next :- (Backend)

Step 1: PostgreSQL/PostGIS tables

sites

villages

instruments

survey_records

zones

uploaded_files

users

sessions

Step 2: APIs

GET /api/sites

GET /api/sites/:id

GET /api/sites/:id/instruments

GET /api/sites/:id/surveys

GET /api/sites/:id/villages

POST /api/login

POST /api/upload

Step 3: Frontend change

Replace:

`fetch("../data/sites.json")`

with:

`fetch("http://localhost:5000/api/sites")`

or for offline local server:

`fetch("/api/sites")`

Everything else can remain similar.

PostgreSQL + PostGIS setup later

When you are ready for backend/database, use these commands.

Open SQL Shell or CMD:

`psql -U postgres`

Create database:

`CREATE DATABASE npp_gis;`

Connect:

`\c npp_gis`

Enable PostGIS:

`CREATE EXTENSION postgis;`

Check:

`SELECT postgis_full_version();`

STEP 9: Create final tables

Run this inside `npp_gis`:

`CREATE TABLE users (`

`id SERIAL PRIMARY KEY,`

`username TEXT UNIQUE NOT NULL,`

`password_hash TEXT NOT NULL,`

role TEXT DEFAULT 'viewer',
);

created_at TIMESTAMP DEFAULT
CURRENT_TIMESTAMP

);

CREATE TABLE sites (

id SERIAL PRIMARY KEY,

name TEXT NOT NULL,

state TEXT,

zone TEXT,

description TEXT,

latitude DOUBLE PRECISION,

longitude DOUBLE PRECISION,

geom GEOMETRY(Point, 4326)

);

CREATE TABLE instruments (

id SERIAL PRIMARY KEY,

site_id INTEGER REFERENCES sites(id)
ON DELETE CASCADE,

name TEXT NOT NULL,

type TEXT,

status TEXT,

description TEXT

CREATE TABLE survey_data (

id SERIAL PRIMARY KEY,

site_id INTEGER REFERENCES sites(id)
ON DELETE CASCADE,

survey_name TEXT,

survey_date DATE,

summary TEXT,

file_path TEXT

);

CREATE TABLE villages (

id SERIAL PRIMARY KEY,

site_id INTEGER REFERENCES sites(id)
ON DELETE CASCADE,

name TEXT NOT NULL,

population INTEGER,

sector TEXT,

distance_km DOUBLE PRECISION,

latitude DOUBLE PRECISION,

longitude DOUBLE PRECISION,

geom GEOMETRY(Point, 4326)

);

```
CREATE TABLE uploaded_files (
  id SERIAL PRIMARY KEY,
  site_id INTEGER REFERENCES sites(id)
ON DELETE CASCADE,
  file_name TEXT,
  file_path TEXT,
  file_type TEXT,
  uploaded_at TIMESTAMP DEFAULT
CURRENT_TIMESTAMP
);
```

STEP 10: Insert sample site data

```
INSERT INTO sites
(name, state, zone, description, latitude,
longitude, geom)
```

```
VALUES
```

```
(
'Sample NPP Site 1',
'Maharashtra',
'Western Zone',
'Placeholder site. To be replaced with
official NPP data.',
19.83,
72.66,
```

```
ST_SetSRID(ST_MakePoint(72.66, 19.83),
4326)
```

```
),
```

```
(
```

```
'Sample NPP Site 2',
```

```
'Tamil Nadu',
```

```
'Southern Zone',
```

```
'Placeholder site. To be replaced with
official NPP data.',
```

```
8.17,
```

```
77.71,
```

```
ST_SetSRID(ST_MakePoint(77.71, 8.17),
4326)
```

```
);
```

Important:

```
ST_MakePoint(longitude, latitude)
```

STEP 11: Insert sample village data

```
INSERT INTO villages
```

```
(site_id, name, population, sector,
distance_km, latitude, longitude, geom)
```

```
VALUES
```

```
(
```

```
1,
```

```
'Village A',
```

```
1200,  
'NE',  
12,  
19.90,  
72.75,  
ST_SetSRID(ST_MakePoint(72.75, 19.90),  
4326)  
  
,  
(  
1,  
'Village B',  
800,  
'E',  
18,  
19.82,  
72.86,  
ST_SetSRID(ST_MakePoint(72.86, 19.82),  
4326)  
  
,  
(  
1,  
'Village C',  
1500,  
'SW',
```

```
25,  
19.65,  
72.50,  
ST_SetSRID(ST_MakePoint(72.50, 19.65),  
4326)  
  
);
```

STEP 12: Insert sample instruments

```
INSERT INTO instruments  
(site_id, name, type, status, description)  
VALUES  
(  
1,  
'Radiation Monitor',  
'Radiation',  
'Active',  
'Used for radiation level monitoring.'  
  
,  
(  
1,  
'Wind Sensor',  
'Meteorological',  
'Active',
```

'Used for wind speed and direction monitoring.'

);

STEP 13: Insert sample survey

INSERT INTO survey_data

(site_id, survey_name, survey_date,
summary, file_path)

VALUES

(

1,

'Population Survey',

'2026-06-01',

'Sample population survey data for nearby
settlements.',

NULL

);

STEP 14: Backend setup later

Inside backend folder:

npm init -y

npm install express pg cors multer
express-session connect-pg-simple bcrypt

Create:

backend/

├── server.js

├── db.js

├── uploads/

└── routes/

├── sites.routes.js

├── auth.routes.js

└── upload.routes.js

STEP 15: backend/db.js

const { Pool } = require("pg");

const pool = new Pool({

user: "postgres",

host: "localhost",

database: "npp_gis",

password: "YOUR_PASSWORD",

port: 5432

});

module.exports = pool;

STEP 16: backend/server.js

```
const express = require("express");  
  
const cors = require("cors");  
  
const session = require("express-session");  
  
const path = require("path");
```

```
const sitesRoutes =  
require("../routes/sites.routes");
```

```
const uploadRoutes =  
require("../routes/upload.routes");
```

```
const app = express();
```

```
app.use(cors({  
  
  origin: true,  
  
  credentials: true  
}));
```

```
app.use(express.json());
```

```
app.use(session({  
  
  secret: "npp-gis-secret",  
  
  resave: false,  
  
  saveUninitialized: false,
```

```
  cookie: {  
  
    maxAge: 1000 * 60 * 60  
  
  }  
}));
```

```
app.use("/uploads",  
express.static(path.join(__dirname,  
"uploads")));
```

```
app.get("/api/health", (req, res) => {  
  
  res.json({ status: "Backend running" });  
  
});
```

```
app.use("/api/sites", sitesRoutes);  
  
app.use("/api/upload", uploadRoutes);
```

```
app.listen(5000, () => {  
  
  console.log("Server running on  
http://localhost:5000");  
  
});
```

STEP 17: backend/routes/sites.routes.js

```
const express = require("express");  
  
const pool = require("../db");
```

```

const router = express.Router();

router.get("/", async (req, res) => {
  try {
    const result = await pool.query(`
      SELECT id, name, state, zone,
      description, latitude, longitude
      FROM sites
      ORDER BY id
    `);

    res.json(result.rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

router.get("/:id", async (req, res) => {
  try {
    const site = await pool.query(
      "SELECT * FROM sites WHERE id = $1",
      [req.params.id]
    );
  }
});

```

```

);

const instruments = await pool.query(
  "SELECT * FROM instruments WHERE
  site_id = $1",
  [req.params.id]
);

const surveys = await pool.query(
  "SELECT * FROM survey_data
  WHERE site_id = $1",
  [req.params.id]
);

const villages = await pool.query(
  "SELECT * FROM villages WHERE
  site_id = $1",
  [req.params.id]
);

res.json({
  site: site.rows[0],
  instruments: instruments.rows,
  surveys: surveys.rows,
  villages: villages.rows
});

```



```

    });

    } catch (err) {

        res.status(500).json({ error: err.message
    });

    }

});

```

```

module.exports = router;

```

STEP 18: **backend/routes/upload.routes.js**

```

const express = require("express");

const multer = require("multer");

const pool = require("../db");

const router = express.Router();

const storage = multer.diskStorage({

    destination: function (req, file, cb) {

        cb(null, "uploads/");

    },

    filename: function (req, file, cb) {

        const uniqueName = Date.now() + "-" +
file.originalname;

        cb(null, uniqueName);

```

```

    }

});

const upload = multer({ storage });

router.post("/", upload.single("file"), async
(req, res) => {

    try {

        const { site_id } = req.body;

        const result = await pool.query(

            `

            INSERT INTO uploaded_files

            (site_id, file_name, file_path, file_type)

            VALUES ($1, $2, $3, $4)

            RETURNING *

            `,

            [

                site_id,

                req.file.originalname,

                req.file.path,

                req.file.mimetype

            ]

        );

```

```
});
```

```
res.json(result.rows[0]);
```

```
} catch (err) {
```

```
  res.status(500).json({ error: err.message  
});
```

```
}
```

```
module.exports = router;
```